

DOSSIER PROJET

*Titre professionnel (DWWM)
Développeur Web et Web Mobile*

RankLobby



Rencontre des joueurs de ton niveau

Viti Théo

LaPlateforme 2025 – 2026

SOMMAIRE

- 01 INTRODUCTION
- 02 COMPÉTENCES MISES EN ŒUVRE
- 03 RÉSUMÉ DU PROJET
- 04 CONCEPTION FONCTIONNELLE
- 05 CONCEPTION VISUELLE
- 06 CONCEPTION TECHNIQUE
- 07 ARCHITECTURE DE L'APPLICATION
- 08 DÉVELOPPEMENT
- 09 DÉPLOIEMENT ET CONTENEURISATION
- 10 PLAN DE TESTS ET JEU D'ESSAI
- 11 CONCLUSION

01 INTRODUCTION

Ce dossier présente le projet « **RankLobby** » qui a été réalisé dans le cadre d'un travail individuel au sein de la formation de développeur web et web mobile, qui s'est déroulée en présentiel à La Plateforme de Martigues lors de la session 2025/2026.

Cette formation de Développeur Web et Web Mobile est la première étape d'une reconversion professionnelle, ayant commencé par un parcours littéraire. Cette démarche s'inscrit dans un projet à plus long terme, visant à intégrer un CDA en alternance et d'acquérir une expérience professionnelle.

RankLobby est une application web full-stack dédiée à la recherche de coéquipiers pour LE jeu vidéo PC **League of Legends**. Son objectif est de faciliter les rencontres entre joueurs de niveau similaire afin de constituer des équipes durables et d'améliorer l'expérience de jeu. Le projet répond à une problématique concrète : aujourd'hui, la majorité des joueurs utilisent des serveurs Discord pour recruter des partenaires, une solution peu adaptée à la recherche de profils précis et au suivi d'une communauté sur le long terme. RankLobby propose une plateforme centralisant la création de profils, le matchmaking, la gestion d'amis ainsi que des outils de modération au sein d'une interface unique.

02 COMPÉTENCES MISES EN ŒUVRE

Les compétences du référentiel couvertes par le projet sont :

Pour l'activité type 1 : « Développer la partie front-end d'une application web ou web mobile sécurisée » :

Maquetter des interfaces utilisateur web ou web mobile

J'ai conçu les wireframes et les maquettes de RankLobby avec Figma, selon une approche mobile-first avant leur adaptation au format desktop. J'ai défini les parcours utilisateurs, l'organisation des interfaces et la charte graphique de l'application. Les maquettes respectent les principes d'expérience utilisateur, les recommandations d'accessibilité (RGAA/WCAG) ainsi que les obligations liées au RGPD avec l'intégration des pages légales.

Réaliser des interfaces utilisateur statiques web ou web mobile

J'ai développé l'ensemble des interfaces en HTML5, CSS3 et Thymeleaf. L'application suit une architecture responsive mobile-first afin de s'adapter aux différents formats d'écran (min-width). J'ai privilégié un HTML sémantique, des composants réutilisables (header, footer, cartes) et une organisation modulaire des feuilles de style pour faciliter la maintenance du projet.

Développer la partie dynamique des interfaces utilisateur web ou web mobile

J'ai développé les interactions entre les interfaces et le serveur avec Spring Boot et Thymeleaf. Les données affichées (profils, recherche de joueurs, matchmaking, liste d'amis, administration) sont générées dynamiquement depuis le backend. Les formulaires utilisent les **validations HTML** ainsi que les **contrôles réalisés côté serveur** afin de garantir la sécurité et la cohérence des données.

Pour l'activité type 2 : « Développer la partie back-end d'une application web ou web mobile sécurisée »

Mettre en place une base de données relationnelle et non relationnelle

J'ai conçu le modèle de données de l'application à partir de la méthode Merise : MCD, MLD et MPD. J'utilise MySQL pour les données relationnelles (utilisateurs, profils, relations, signalements...) et MongoDB pour le stockage de données non relationnelles. Les tables sont reliées par des clés primaires et étrangères afin de garantir l'intégrité des données

Développer des composants d'accès aux données SQL et NoSQL

J'ai développé les composants d'accès aux données avec **Spring Data JPA** et **Hibernate** pour la base de données **MySQL**, ainsi que des **MongoRepository** pour **MongoDB**. Les accès aux données sont organisés dans des **Repository**, permettant d'isoler la logique de persistance des traitements métier et de faciliter la maintenance de l'application. Cette organisation simplifie les opérations de consultation, de création, de modification et de suppression des données tout en respectant l'architecture MVC du projet.

Développer des composants métier coté serveur d'une application web sécurisée

J'ai développé les traitements métier avec une architecture MVC composée des couches Controller, Service, Repository et DTO. Les composants métier gèrent notamment l'inscription, l'authentification, les profils utilisateurs, le matchmaking, la gestion des amis, les signalements et les fonctionnalités d'administration. Les validations sont réalisées côté serveur et les mots de passe sont sécurisés avec Spring Security.

Documenter le déploiement d'une application dynamique web ou web mobile

J'ai préparé le déploiement de l'application à l'aide de **Docker** afin de faciliter son installation dans un environnement identique à celui de développement. Le projet est versionné avec **Git** et **GitHub**, et j'ai documenté son architecture, son installation, les prérequis techniques ainsi que les principales étapes nécessaires à sa mise en production.

03 RÉSUMÉ DU PROJET

RankLobby est une plateforme web dédiée à la mise en relation de joueurs de jeux vidéo en fonction de leur rang sur le jeu d'équipe très populaire « League of Legends », de leurs objectifs et de leur style de jeu. Elle vise à faciliter la création de groupes de joueurs compatibles pour des parties classées ou occasionnelles, tout en améliorant la qualité des matchs et l'expérience sociale en ligne.

L'application est une proposition de solution à une demande déjà existante :



Exemple d'un produit presque similaire sur un serveur discord

Orientée matchmaking, l'application permet :

- aux joueurs de créer un profil personnalisé, de renseigner leur rang ainsi que leurs préférences de jeu (compétitif ou occasionnel)
- la vérification du profil de jeu via l'API officielle de Riot Games
- la mise en relation sécurisée entre joueurs, un système d'amitié, complété par un système de commentaires permettant de recommander les joueurs les plus sociables

FONCTIONNALITÉS

Un utilisateur non connecté peut :

- Avoir accès à la page d'accueil (découvrir le concept).
- Créer un compte à l'aide du formulaire d'inscription et se connecter et s'authentifier.
- Consulter les conditions générales et politiques d'accès aux données personnelles

Un utilisateur connecté peut :

- créer et modifier son profil (username, avatar, message de statut), ainsi que son profil de jeu League of Legends (pseudonyme sur le jeu, rang, rôle principal, mode de recherche : casual / ranked / les deux)
- demander à faire vérifier son rang via l'API officielle Riot Games
- consulter et explorer les profils d'autres joueurs via matchmaking selon le rang pour le mode compétitif ou liste publique (casual uniquement)
- envoyer, recevoir, accepter ou refuser des invitations de mise en relation
- accéder à un système de matchmaking basé sur le niveau (écart de rang limité)
- consulter les profils incluant commentaires
- ajouter, supprimer ou bloquer des joueurs dans sa friendlist
- signaler un joueur via son profil

Un administrateur peut :

- gérer les utilisateurs (avoir accès aux profils, créer, modifier, supprimer, ban / unban)
 - consulter la liste des signalements et modérer les commentaires
-

CAS D'USAGE PRINCIPAUX

- inscription / connexion / déconnexion
- gestion du profil joueur et des données de jeu
- vérification du rang/profil via API Riot Games
- matchmaking basé sur le niveau du joueur
- gestion des invitations (envoyer / accepter / refuser / annuler)
- révélation du pseudo en jeu après validation mutuelle
- système social (amis, blocage, commentaires, signalement)
- administration et modération de la plateforme

04 CONCEPTION FONCTIONNELLE

Expression des besoins : cahier des charges

Pour initier la conception du projet, j'ai élaboré un cahier des charges fictif définissant l'objectif de l'application ainsi que les fonctionnalités attendues. Celles-ci ont ensuite été organisées selon la méthode MoSCoW, qui permet de hiérarchiser les exigences en quatre catégories : « Must », « Should », « Could » et « Won't », afin d'optimiser la priorisation et de structurer efficacement le développement.

PROJET	Créer une plateforme de mise en relation de joueurs de jeux vidéo par rang et objectifs
Objectif	Application web permettant aux joueurs de créer un profil de jeu vérifié, de trouver des coéquipiers de niveau compatible via un système de matchmaking, et de gérer leurs relations sociales (amis, invitations, avis) au sein d'une communauté.

MoSCoW Method :

Must have	Ce que l'on considère comme indispensable dans le projet.
Should have	Ce que l'on souhaite intégrer, mais dont l'absence resterait acceptable.
Could have	Les éléments secondaires qui apporteraient une valeur ajoutée si le temps de développement le permet.
Won't have	Les fonctionnalités exclues de cette version, mais conservées pour d'éventuelles évolutions futures.

FONCTIONNALITÉS — EN TANT QU'UTILISATEUR VISITEUR (NON CONNECTÉ)

Fonctionnalité	Info supplémentaire	Priorité
Découvrir concept (home)	Je peux découvrir le concept et je suis incité à m'inscrire	Must have
Consulter les pages légales et informatives	Accès aux pages contact, mentions légales et politique de confidentialité	Must have
M'inscrire	Création de compte via email et mot de passe	Must have
Me connecter	Authentification sécurisée via identifiants (email + mot de passe)	Must have

FONCTIONNALITÉS — EN TANT QU'UTILISATEUR CONNECTÉ (PROFIL)

Fonctionnalité	Info supplémentaire	Priorité
Créer/modifier mon profil	Je peux renseigner mon username, avatar, message de statut (sans y inclure mon pseudo en jeu)	Must have
Renseigner mes données de jeu	Je peux indiquer mon rang, mon rôle principal sur League of Legends	Must have
Choisir mon mode de recherche	Je peux définir si je cherche du casual, du ranked, ou les deux	Must have
Faire vérifier mon profil LoL	Je peux faire valider mon rang via l'API officielle Riot Games	Should have
Renseigner un rôle secondaire	Je peux ajouter jusqu'à 3 champions pour un sub role optionnel	Won't have

FONCTIONNALITÉS — EN TANT QU'UTILISATEUR CONNECTÉ (MATCHMAKING)

Fonctionnalité	Info supplémentaire	Priorité
Explorer les joueurs	Je peux rechercher un utilisateur manuellement ou consulter la liste publique des joueurs en casual	Must have
Lancer un matchmaking	Je peux demander une liste de joueurs compatibles avec mon rang (-/+ 1 palier), envoyer une invitation ou consulter leur profil depuis cette liste	Must have
Gérer mes invitations	Je peux consulter ma boîte de réception, recevoir, accepter, refuser ou annuler une invitation	Must have
Voir le pseudo en jeu à l'acceptation	À l'acceptation d'une invitation, je vois le pseudo en jeu du joueur pour l'ajouter directement	Should have

FONCTIONNALITÉS — EN TANT QU'UTILISATEUR CONNECTÉ (SOCIAL)

Fonctionnalité	Info supplémentaire	Priorité
Gérer ma liste d'amis	Je peux supprimer un ami ou envoyer une demande d'ami directement depuis un profil (sans passer par le matchmaking)	Must have
Consulter mon historique de rencontres	Je peux voir la liste des joueurs rencontrés (profil + date d'acceptation)	Could Have

Bloquer un utilisateur	Je peux bloquer un profil, ce qui annule les invitations en cours et empêche tout futur contact	Should have
Signaler un joueur	Je peux signaler un profil pour comportement inapproprié	Should have
Noter un joueur	Je peux laisser une note (1-5) et un commentaire sur un profil	Could have

FONCTIONNALITÉS — EN TANT QU'ADMIN

Fonctionnalité	Info supplémentaire	Priorité
Gérer les utilisateurs	Je peux bannir/débannir un utilisateur, modérer les commentaires	Must have
Gérer les signalements	Je peux consulter et gérer les signalements	Could have

05 CONCEPTION VISUELLE

Logo



Logo-sans-background



Logo + catchline

Le logo reflète l'identité de RankLobby en combinant les codes visuels de *Discord* et de *League of Legends*. Il symbolise à la fois l'aspect communautaire de la plateforme et son orientation compétitive, tout en conservant un design simple, facilement identifiable et réutilisable sur les différents supports.

Palette de couleurs

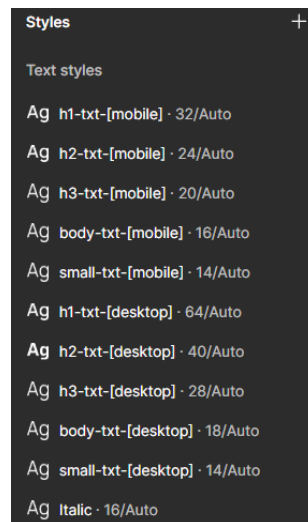


Couleurs principales | secondaires | d'accents | fonctionnelles

La palette de couleurs a été conçue pour offrir une interface sobre, moderne et lisible. Les tons bleus sont utilisés pour les surfaces et la navigation, tandis que le doré met en valeur les éléments importants et rappelle l'univers de League of Legends. Les couleurs fonctionnelles (succès, avertissement et erreur) facilitent également le retour visuel auprès de l'utilisateur.

Typographie :

Afin de garantir la cohérence visuelle de l'application, j'ai créé des **styles typographiques réutilisables** dans Figma. Cette organisation facilite la conception des interfaces, assure une hiérarchie visuelle homogène et simplifie les futures évolutions du projet. La police **Manrope** est utilisée pour l'ensemble de l'application. Pour les rares textes en italique, j'utilise **Inter**, Manrope ne proposant pas de variante italique.

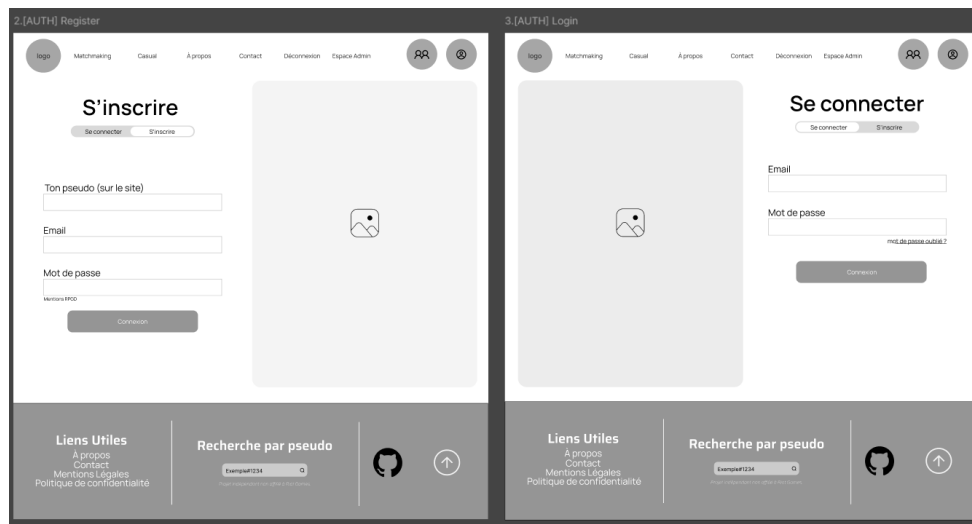


Mes styles réutilisables pour le texte

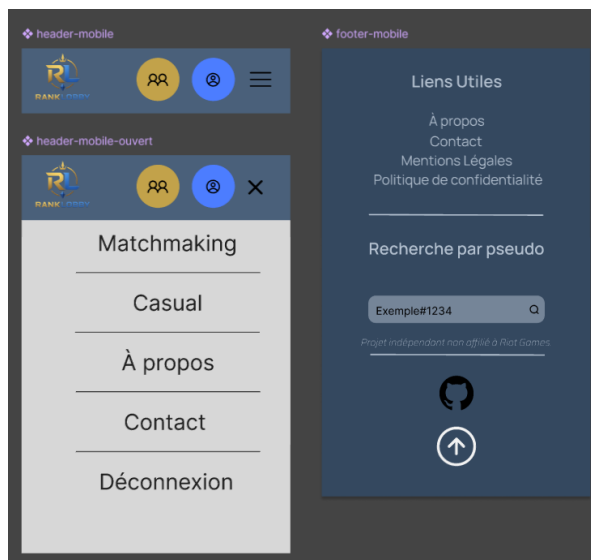
Style	Mobile	Desktop	Police
H1	32 px	64 px	Manrope SemiBold
H2	24 px	40 px	Manrope SemiBold
H3	20 px	28 px	Manrope Regular
Body	16 px	18 px	Manrope Regular
Small	14 px	16 px	Manrope Regular
Italique	16 px	18 px	Inter Italic

Wireframes et Maquettes :

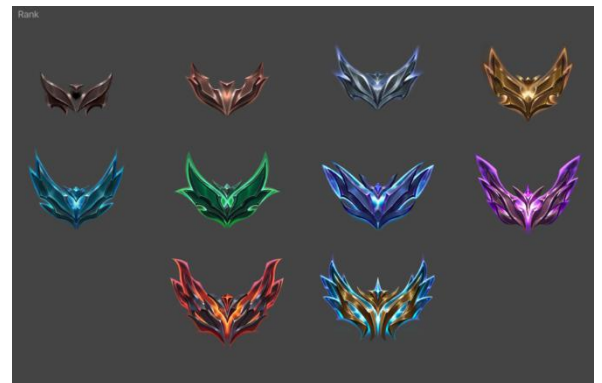
En adoptant une approche **mobile-first**, j'ai conçu l'ensemble des wireframes et des maquettes de l'application sur **Figma** pour les versions mobiles et desktop. J'ai également créé des **composants réutilisables** (boutons, gamercard, header, barre de navigation, footer, etc.) afin d'assurer la cohérence graphique, de faciliter la maintenance des maquettes et d'accélérer la conception des différentes interfaces.



Wireframe Desktop pour les pages d'authentification

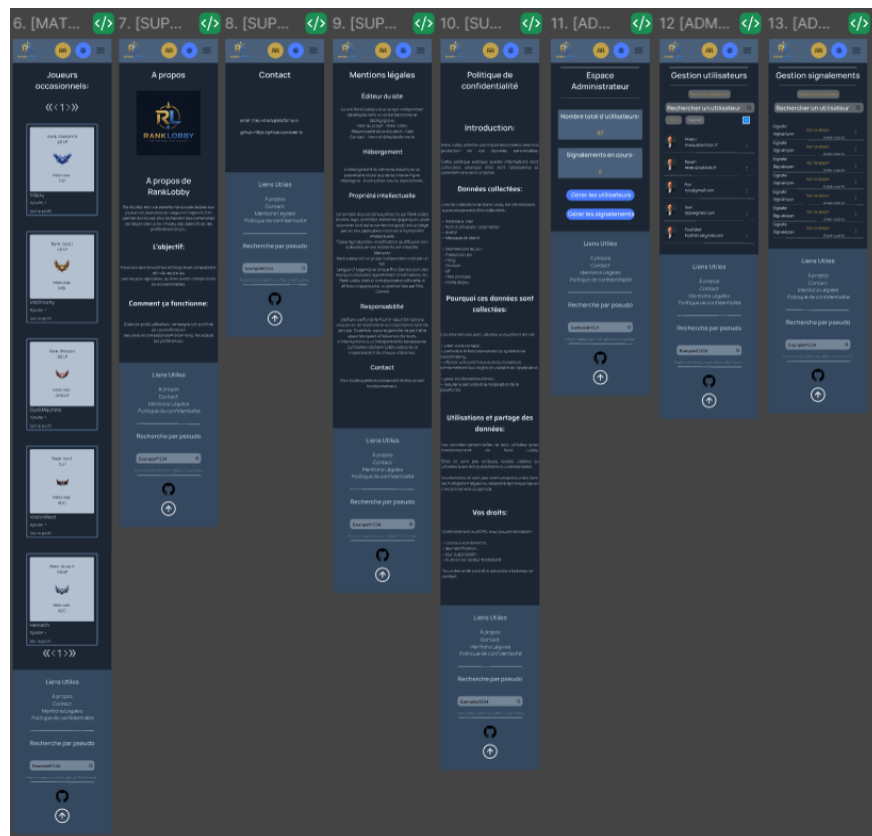


Components mobile :



Asset : les icones pour les rangs fournis par Riot Games (entreprise détenant League of Legends)





Maquettes/Mockups des pages mobile

06 CONCEPTION TECHNIQUE

ENVIRONNEMENT TECHNIQUE



Backend

Le backend de RankLobby repose sur **Java 17** et **Spring Boot**. J'ai choisi ce framework pour son écosystème complet, qui facilite le développement d'une application web sécurisée. Il intègre notamment un **serveur Tomcat embarqué**, une **configuration automatisée** et **l'injection de dépendances** grâce aux annotations `@RestController`, `@Controller`, `@Service` et `@Repository`, permettant de structurer clairement les différentes couches de l'application.

```
@Repository 4 usages kael  
public interface UserRepository extends JpaRepository<User, Long> {
```

UserRepository qui étend l'interface JpaRepository

L'accès aux données est simplifié grâce à **Spring Data JPA** et **Hibernate**, qui permettent de manipuler les entités Java sans écrire directement de requêtes SQL pour les opérations courantes. La sécurité est assurée par **Spring Security**, utilisé pour l'authentification, le contrôle des accès et le hachage des mots de passe avec **BCrypt**.

Frontend

Le frontend est développé avec **Thymeleaf**, **HTML5** et **CSS3**. Thymeleaf permet de générer des pages HTML dynamiques directement à partir des données du backend Spring Boot. Les interfaces ont été conçues selon une approche mobile-first, puis adaptées aux écrans plus larges grâce à des media queries utilisant principalement des breakpoints en min-width. L'application utilise un layout commun ainsi que des fragments réutilisables (header, footer) afin d'assurer une interface cohérente et de faciliter la maintenance.

```
<section class="friends-list" aria-label="Liste des amis">
    <article class="friend-card" th:each="friend : ${friends}">
```

Bases de données

RankLobby utilise deux bases de données complémentaires. MySQL stocke les données relationnelles de l'application (utilisateurs, profils de jeu, invitations, relations d'amitié, signalements...). MongoDB est destiné au stockage des avis et commentaires laissés entre joueurs. Ce choix est adapté à des données susceptibles d'évoluer dans leur structure et pouvant contenir un volume important de texte, tout en évitant de complexifier le modèle relationnel principal.

Spring Initializr

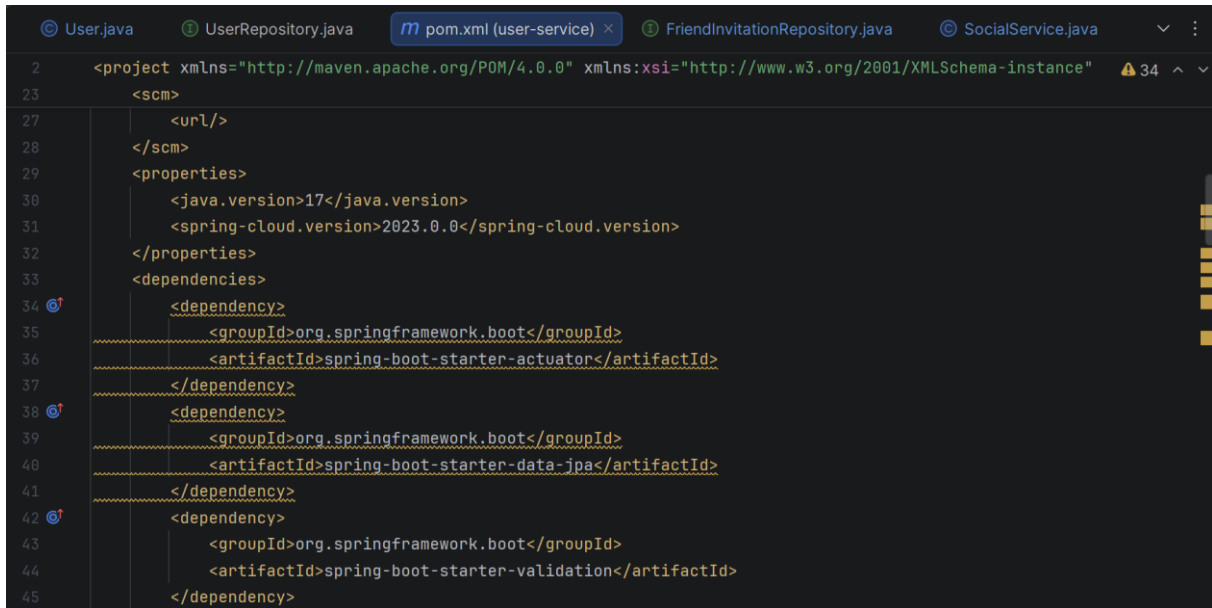
J'ai utilisé Spring Initializr pour créer la structure de base de chaque microservice. Cet outil génère automatiquement un projet Maven configuré avec les dépendances nécessaires, telles que Spring Web, Spring Security, Spring Data JPA, Thymeleaf ou Validation. Cette méthode permet de démarrer rapidement le développement tout en garantissant une architecture cohérente, standardisée et facilement évolutive.

The screenshot shows the Spring Initializr web application interface. On the left, under 'Project', 'Maven' is selected. Under 'Language', 'Java' is selected. Under 'Spring Boot', '3.5.16' is selected. The 'Project Metadata' section shows 'Group' as 'com.ranklobby', 'Artifact' as 'webapp', and 'Package name' as 'com.ranklobby.webapp'. 'Packaging' is set to 'Jar' and 'Configuration' to 'Properties'. On the right, the 'Dependencies' section lists several dependencies: 'Spring Web' (WEB), 'Thymeleaf' (TEMPLATE ENGINES), 'Spring Security' (SECURITY), 'OpenFeign' (SPRING CLOUD ROUTING), 'Consul Discovery' (SPRING CLOUD DISCOVERY), 'Spring Boot Actuator' (OPS), and 'Lombok' (DEVELOPER TOOLS). Each dependency has a brief description.

Initialisation du point d'entrée frontend « webapp » et de ses dépendances

Maven

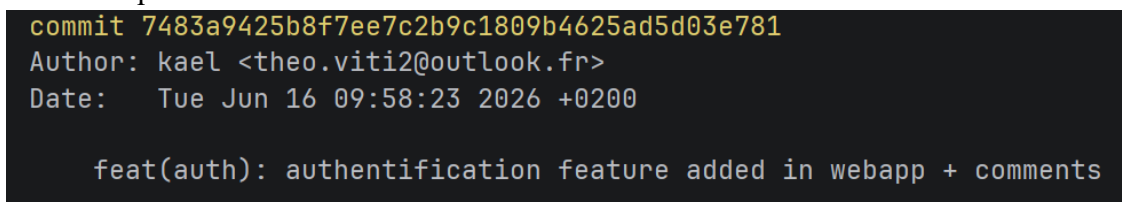
Maven est un outil de gestion qui gère le cycle de vie d'un projet Java, les dépendances et la compilation. Le fichier pom.xml centralise toutes les informations relatives aux bibliothèques, plugins et configurations nécessaires au projet, ce qui permet de reproduire facilement l'environnement de développement et d'automatiser la construction, les tests et le déploiement. De plus, Maven impose une structure de projet standardisée, avec des dossiers dédiés: package ressources pour les fichiers de configuration et ressources statiques, et package test pour les tests unitaires et d'intégration.



```
2 <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
23 <scm>
27 <url/>
28 </scm>
29 <properties>
30 <java.version>17</java.version>
31 <spring-cloud.version>2023.0.0</spring-cloud.version>
32 </properties>
33 <dependencies>
34 <dependency>
35 <groupId>org.springframework.boot</groupId>
36 <artifactId>spring-boot-starter-actuator</artifactId>
37 </dependency>
38 <dependency>
39 <groupId>org.springframework.boot</groupId>
40 <artifactId>spring-boot-starter-data-jpa</artifactId>
41 </dependency>
42 <dependency>
43 <groupId>org.springframework.boot</groupId>
44 <artifactId>spring-boot-starter-validation</artifactId>
45 </dependency>
```

pom.xml (micro-service User)

Git et GitHub sont utilisés pour le versionnement du projet et le suivi des évolutions grâce à un historique de commits.



```
commit 7483a9425b8f7ee7c2b9c1809b4625ad5d03e781
Author: kael <theo.viti2@outlook.fr>
Date: Tue Jun 16 09:58:23 2026 +0200

feat(auth): authentication feature added in webapp + comments
```

Exemple d'un commit Git

Technologies utilisées :

Langages de programmation	Java 17
Langage de balisage, style (Front)	HTML5 / CSS3
Frameworks / Bibliothèques	Spring Boot, Thymeleaf, Feign, Hibernate / JPA
Gestion des dépendances	Maven
Base de données	MySQL, MongoDB
ORM	Spring Data JPA / Hibernate (SQL) Spring Data MongoDB (NoSQL)
Découverte et l'enregistrement dynamique des micro services	Consul
Conteneurisation / Devops	Docker compose
Environnement de développement (IDE)	IntelliJ IDEA
Gestion de version	Git, GitHub
Prototypage / Design UI	Figma
Tests / Qualité	JUnit, Mockito, Postman
Sécurité	Spring Security
Documentation API	Swagger
Configuration distribuée	Spring Cloud Config Server

Justification des choix de base de données

MongoDB : avis et commentaires (données non relationnelles)

J'ai choisi **MongoDB** pour stocker les avis laissés entre joueurs. Ces données sont moins liées au modèle relationnel principal et peuvent contenir du texte libre, une note, une date de création et éventuellement une date de modification. MongoDB permet de stocker ces avis

sous forme de documents souples, tout en gardant une référence vers les utilisateurs MySQL grâce à leurs identifiants.

MySQL : données relationnelles

J'ai choisi **MySQL** pour stocker les données principales de RankLobby, car elles sont fortement relationnelles : utilisateurs, profils de jeu, invitations, relations d'amitié, blocages et signalements. Ces données nécessitent des contraintes fortes comme les clés primaires, les clés étrangères, les valeurs uniques et l'intégrité référentielle. MySQL est donc adapté pour garantir la cohérence des relations entre utilisateurs.

MODÉLISATION ET CONCEPTION DE LA BASE DE DONNÉES

Avant le développement, la base de données a été conçue selon la méthode Merise en réalisant successivement un MCD, un MLD puis un MPD. Cette démarche a permis de définir les entités, leurs relations et les contraintes d'intégrité avant leur implémentation.

MCD

Le **Modèle Conceptuel de Données (MCD)** représente les principales entités du projet ainsi que leurs relations, indépendamment de toute contrainte technique. Il permet de définir les besoins fonctionnels de l'application avant le choix du système de gestion de base de données.

Entité Relation Entité Cardinalité (E1) Cardinalité (E2) Règle métier

Entité E1	Relation	Entité E2	Card. E1	Card. E2	Règle métier
USER	possède	GAMER_PROFILE	0,1	1,1	Un utilisateur peut avoir au maximum un profil de jeu ; chaque profil de jeu appartient à un seul utilisateur.
USER	envoie	FRIEND_INVITATION	0,N	1,1	Un utilisateur peut envoyer plusieurs invitations ; chaque invitation possède un expéditeur.
USER	reçoit	FRIEND_INVITATION	0,N	1,1	Un utilisateur peut recevoir plusieurs invitations ; chaque invitation possède un destinataire.
USER	participe comme user1	FRIENDSHIP	0,N	1,1	Une amitié relie exactement deux utilisateurs distincts.
USER	participe comme user2	FRIENDSHIP	0,N	1,1	Chaque occurrence d'amitié référence un second utilisateur.
USER	bloque	USER_BLOCK	0,N	1,1	Un utilisateur peut bloquer plusieurs utilisateurs.
USER	est bloqué par	USER_BLOCK	0,N	1,1	Chaque blocage concerne exactement un utilisateur bloqué.
USER	signale	REPORT	0,N	1,1	Un utilisateur peut effectuer plusieurs signalements.
USER	est signalé	REPORT	0,N	1,1	Chaque signalement cible exactement un utilisateur.
USER	rédige	REVIEW (MongoDB)	0,N	1,1	Un utilisateur peut rédiger plusieurs avis.
USER	est évalué	REVIEW (MongoDB)	0,N	1,1	Chaque avis référence un profil utilisateur évalué.

MLD

Le **Modèle Logique de Données (MLD)** traduit le MCD en un modèle relationnel. Chaque entité devient une table, les identifiants deviennent des clés primaires et les relations sont matérialisées par des clés étrangères.

user-service

MLD - USER

Champ	Contrainte	Description
id	PK	Identifiant technique
email	UNIQUE NOT NULL	Adresse e-mail, 100 caractères maximum
username	UNIQUE NOT NULL	Pseudo, 3 à 20 caractères, sans espace, unicité sans distinction de casse
password	NOT NULL	Hash BCrypt ; aucun mot de passe en clair
avatar_url	NULL	Chemin public vers l'avatar ; avatar par défaut à la création
status_message	NULL	Texte nettoyé, 255 caractères maximum
role	DEFAULT USER	USER ou ADMIN
is_banned	DEFAULT FALSE	État de bannissement du compte
created_at	NULL	Date de création
updated_at	NULL	Date de dernière modification

MLD - GAMER_PROFILE

Champ	Contrainte	Description
id	PK	Identifiant technique
user_id	FK UNIQUE NOT NULL	Relation 1-1 vers USER
pseudo_in_game	NOT NULL	Riot ID au format Nom#TAG, 3 à 30 caractères
rank_tier	NOT NULL	IRON à CHALLENGER
rank_division	NULL conditionnel	I à IV de Iron à Diamond ; NULL à partir de Master
league_points	NOT NULL	0 à 100 jusqu'à Diamond ; entier positif à partir de Master
game_mode	NOT NULL	CASUAL, RANKED ou BOTH
rank_verified	DEFAULT FALSE	Rang vérifié via l'API Riot
main_role	NOT NULL	TOP, JUNGLE, MID, ADC ou SUPPORT
updated_at	NULL	Date de dernière mise à jour

social-service

MLD - FRIEND_INVITATION

Champ	Contrainte	Description
id	PK	Identifiant technique
sender_id	REF. USER NOT NULL	Identifiant externe de l'expéditeur
receiver_id	REF. USER NOT NULL	Identifiant externe du destinataire
status	NOT NULL DEFAULT PENDING	PENDING, ACCEPTED, REFUSED ou CANCELLED
created_at	NOT NULL	Date de création
(sender_id, receiver_id)	UNIQUE	Une seule occurrence pour un couple orienté

MLD - FRIEND_INVITATION

Champ	Contrainte	Description
id	PK	Identifiant technique
sender_id	REF. USER NOT NULL	Identifiant externe de l'expéditeur
receiver_id	REF. USER NOT NULL	Identifiant externe du destinataire
status	NOT NULL DEFAULT PENDING	PENDING, ACCEPTED, REFUSED ou CANCELLED
created_at	NOT NULL	Date de création
(sender_id, receiver_id)	UNIQUE	Une seule occurrence pour un couple orienté

MLD - FRIENDSHIP

Champ	Contrainte	Description
id	PK	Identifiant technique
user1_id	REF. USER NOT NULL	Premier utilisateur
user2_id	REF. USER NOT NULL	Second utilisateur
accepted_at	NOT NULL	Date d'acceptation
(user1_id, user2_id)	UNIQUE	Une seule amitié pour le couple normalisé

MLD - USER_BLOCK

Champ	Contrainte	Description
id	PK	Identifiant technique
blocker_id	REF. USER NOT NULL	Utilisateur à l'origine du blocage
blocked_id	REF. USER NOT NULL	Utilisateur bloqué
created_at	NOT NULL	Date de création
(blocker_id, blocked_id)	UNIQUE	Un blocage unique par couple orienté

MLD - REPORT

Champ	Contrainte	Description
id	PK	Identifiant technique
reporter_id	REF. USER NOT NULL	Auteur du signalement
reported_id	REF. USER NOT NULL	Utilisateur signalé
reason	NOT NULL	Motif de 10 à 255 caractères
created_at	NOT NULL	Date de création
(reporter_id, reported_id)	UNIQUE	Un signalement unique par couple orienté

MLD - REVIEW (MongoDB, fonctionnalité prévue)

Champ	Type / contrainte	Description
_id	ObjectId / String	Identifiant du document MongoDB
profile_user_id	Long NOT NULL	Identifiant externe de l'utilisateur évalué
author_user_id	Long NOT NULL	Identifiant externe de l'auteur
comment	String NOT NULL	Commentaire de 10 à 255 caractères
created_at	Date	Date de création

MPD

Le **Modèle Physique de Données (MPD)** décrit l'implémentation réelle de la base de données dans MySQL. Il précise les types de données, les contraintes (PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL) ainsi que les index nécessaires aux performances.

user-service

MySQL - user-service - users

Colonne	Type	Contraintes
id	BIGINT	PK AUTO_INCREMENT
email	VARCHAR(100)	NOT NULL UNIQUE
username	VARCHAR(20)	NOT NULL UNIQUE ; comparaison insensible à la casse
password	VARCHAR(255)	NOT NULL
avatar_url	VARCHAR(255)	NULL
status_message	VARCHAR(255)	NULL
role	VARCHAR(20)	DEFAULT 'USER'
is_banned	BOOLEAN / TINYINT(1)	DEFAULT FALSE
created_at	TIMESTAMP	NULL
updated_at	TIMESTAMP	NULL

MySQL - user-service - gamer_profile

Colonne	Type	Contraintes
id	BIGINT	PK AUTO_INCREMENT
user_id	BIGINT	NOT NULL UNIQUE, FK users(id)
pseudo_in_game	VARCHAR(30)	NOT NULL
rank_tier	VARCHAR(20)	NOT NULL
rank_division	VARCHAR(4)	NULL
league_points	INT	NOT NULL, valeur positive
game_mode	VARCHAR(20)	NOT NULL
rank_verified	BOOLEAN / TINYINT(1)	DEFAULT FALSE
main_role	VARCHAR(20)	NOT NULL
updated_at	TIMESTAMP	NULL

social-service

MySQL - social-service - friend_invitation

Colonne	Type	Contraintes
id	BIGINT	PK AUTO_INCREMENT
sender_id	BIGINT	NOT NULL, référence externe USER
receiver_id	BIGINT	NOT NULL, référence externe USER
status	VARCHAR(20)	NOT NULL DEFAULT 'PENDING'
created_at	TIMESTAMP	NOT NULL
UNIQUE	(sender_id, receiver_id)	Contrainte d'unicité

MySQL - social-service - friendship

Colonne	Type	Contraintes
id	BIGINT	PK AUTO_INCREMENT
user1_id	BIGINT	NOT NULL, référence externe USER
user2_id	BIGINT	NOT NULL, référence externe USER
accepted_at	TIMESTAMP	NOT NULL
UNIQUE	(user1_id, user2_id)	Contrainte d'unicité

MySQL - social-service - user_block

Colonne	Type	Contraintes
id	BIGINT	PK AUTO_INCREMENT
blocker_id	BIGINT	NOT NULL, référence externe USER
blocked_id	BIGINT	NOT NULL, référence externe USER
created_at	TIMESTAMP	NOT NULL
UNIQUE	(blocker_id, blocked_id)	Contrainte d'unicité

MySQL - social-service - report

Colonne	Type	Contraintes
id	BIGINT	PK AUTO_INCREMENT
reporter_id	BIGINT	NOT NULL, référence externe USER
reported_id	BIGINT	NOT NULL, référence externe USER
reason	VARCHAR(255)	NOT NULL
created_at	TIMESTAMP	NOT NULL
UNIQUE	(reporter_id, reported_id)	Contrainte d'unicité

MongoDB – social service – reviews

Champ	Type	Contraintes
_id	ObjectId / String	Identifiant du document
profile_user_id	Long	NOT NULL, référence externe USER
author_user_id	Long	NOT NULL, référence externe USER
comment	String	NOT NULL, 10 à 255 caractères
created_at	Date	Date de création

Remarque JPA / Hibernate : dans cette version, les scripts SQL ne sont pas écrits manuellement. Les tables MySQL sont générées à partir des entités Java annotées avec @Entity, @Id, @GeneratedValue, @Column, @OneToOne, @ManyToOne ou @OneToMany. Les accès aux données sont réalisés via JpaRepository pour MySQL et MongoRepository pour MongoDB.

L'accès aux données est assuré par des Repository. Les entités relationnelles utilisent des interfaces héritant de JpaRepository, tandis que les données MongoDB sont gérées à l'aide de MongoRepository. Cette séparation isole la logique de persistance des traitements métier réalisés dans les services et facilite la maintenance de l'application.

Sécurité des accès : un utilisateur MySQL est dédié pour gérer la base de données de l'application, avec uniquement les droits nécessaires, afin d'éviter l'usage du compte administrateur root.

```

1 • CREATE USER 'ranklobbyAdmin'@'localhost' IDENTIFIED BY 'RkLobby#2026';
2
3 • GRANT SELECT, INSERT, UPDATE, DELETE, CREATE
4   ON ranklobby.*
5   TO 'ranklobbyAdmin'@'localhost';
6
7 • FLUSH PRIVILEGES;

```

Les accès via Spring Data JPA reposent sur des requêtes préparées pour les opérations courantes, ce qui limite les risques d'injection SQL.

Les formulaires utilisent une double validation : une première validation côté client grâce aux attributs HTML (required, maxlength, type, etc.), puis une validation côté serveur avec **Jakarta Validation** (@Valid, @NotBlank, @Email, @Size...), garantissant l'intégrité des données même si les contrôles HTML sont contournés.

```

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

@NotBlank(message = "L'email est obligatoire.")
@email(message = "Format d'email invalide.")
@Size(max = 100)
@Column(nullable = false, unique = true)
private String email;

@NotBlank(message = "Le nom d'utilisateur est obligatoire.")
@Size(min = 3, max = 20, message = "Le nom d'utilisateur doit contenir entre 3 et 30 caractères.")
@Column(nullable = false, unique = true)
private String username;

@NotBlank(message = "Le mot de passe est obligatoire.")
@Size(min = 8, max = 20, message = "Le mot de passe doit contenir au moins 8 caractères.")
@Pattern(
    regexp = "^(?=.*[a-z])(?=.*[A-Z])(?=.*\\d).+$",
    message = "Le mot de passe doit contenir au moins une majuscule, une minuscule et un chiffre."
)
@Column(nullable = false)
private String password;

@Size(max = 255)
private String avatarUrl;

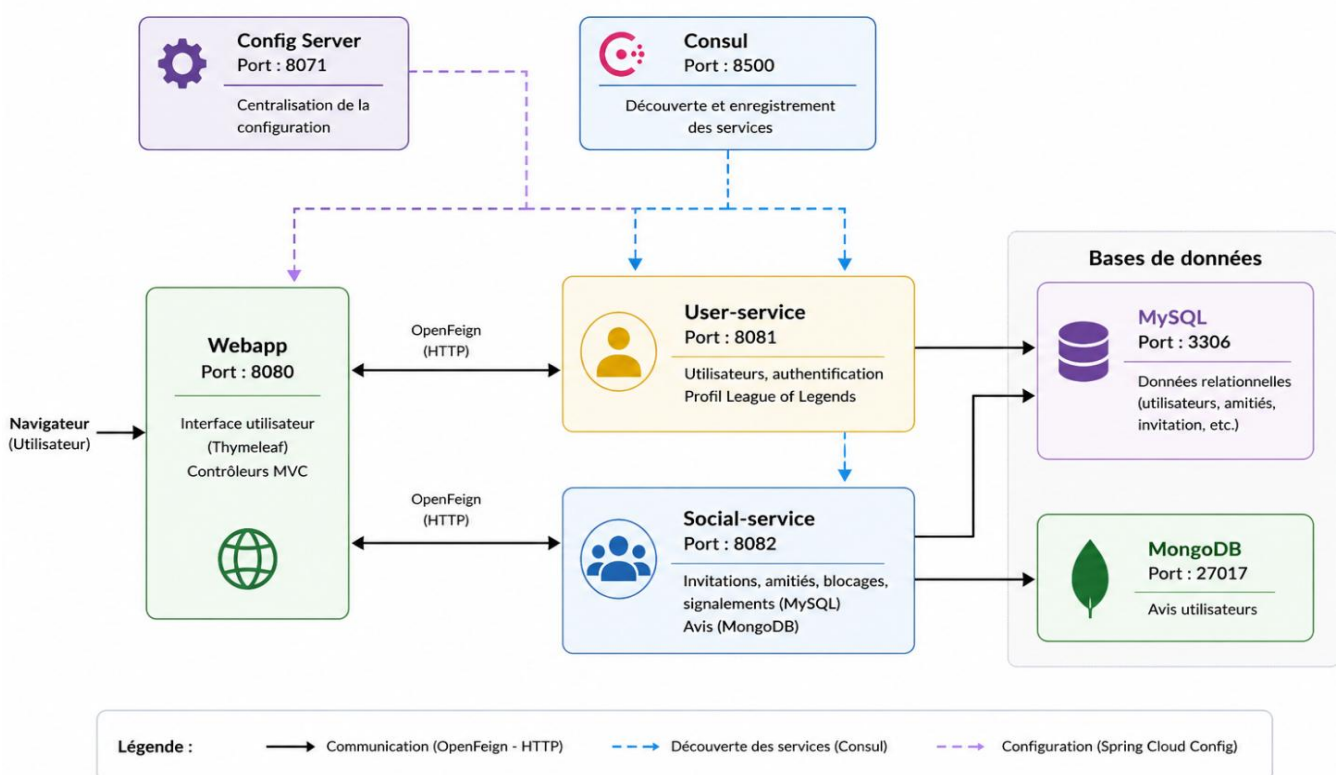
```

Entité User + annotations validations backend

07 ARCHITECTURE DE L'APPLICATION

Présentation de l'architecture

RankLobby repose sur une architecture en **microservices**. L'application est composée d'une interface web Thymeleaf (webapp) et de deux microservices métiers (user-service, social-service). Le découpage permet de séparer les responsabilités, de faciliter la maintenance et de limiter les dépendances entre les différentes fonctionnalités.



Composants de l'application

Composant	Port	Rôle
Webapp	8080	<i>Interface utilisateur, contrôleurs MVC et vues Thymeleaf.</i>
User-service	8081	<i>Gestion des utilisateurs, authentification et profil League of Legends (MySQL).</i>
Social-service	8082	<i>Gestion des invitations, amitiés, blocages, signalements (MySQL) et avis (MongoDB).</i>
Config Server	8071	<i>Centralisation de la configuration des microservices.</i>
Consul	8500	<i>Découverte et enregistrement automatique des services.</i>
MySQL	3306	<i>Stockage des données relationnelles.</i>
MongoDB	27017	<i>Stockage des avis utilisateurs.</i>

Communication entre les services

Le webapp constitue l'unique point d'entrée de l'application. Les échanges avec les microservices s'effectuent via **OpenFeign**, une bibliothèque Spring permettant d'appeler les API des autres services sans écrire manuellement les requêtes HTTP. Les services sont localisés automatiquement grâce à Consul ce qui évite de configurer manuellement leurs adresses réseau.

```

package com.ranklobby.webapp.client;

import ...
@FeignClient(name = "user-service", contextId = "userClient") no usages 1 implementation kael
public interface UserClient {

    @PostMapping("/api/users/register") no usages kael
    UserSessionDto register(@RequestBody UserRegisterDto dto);
}

```

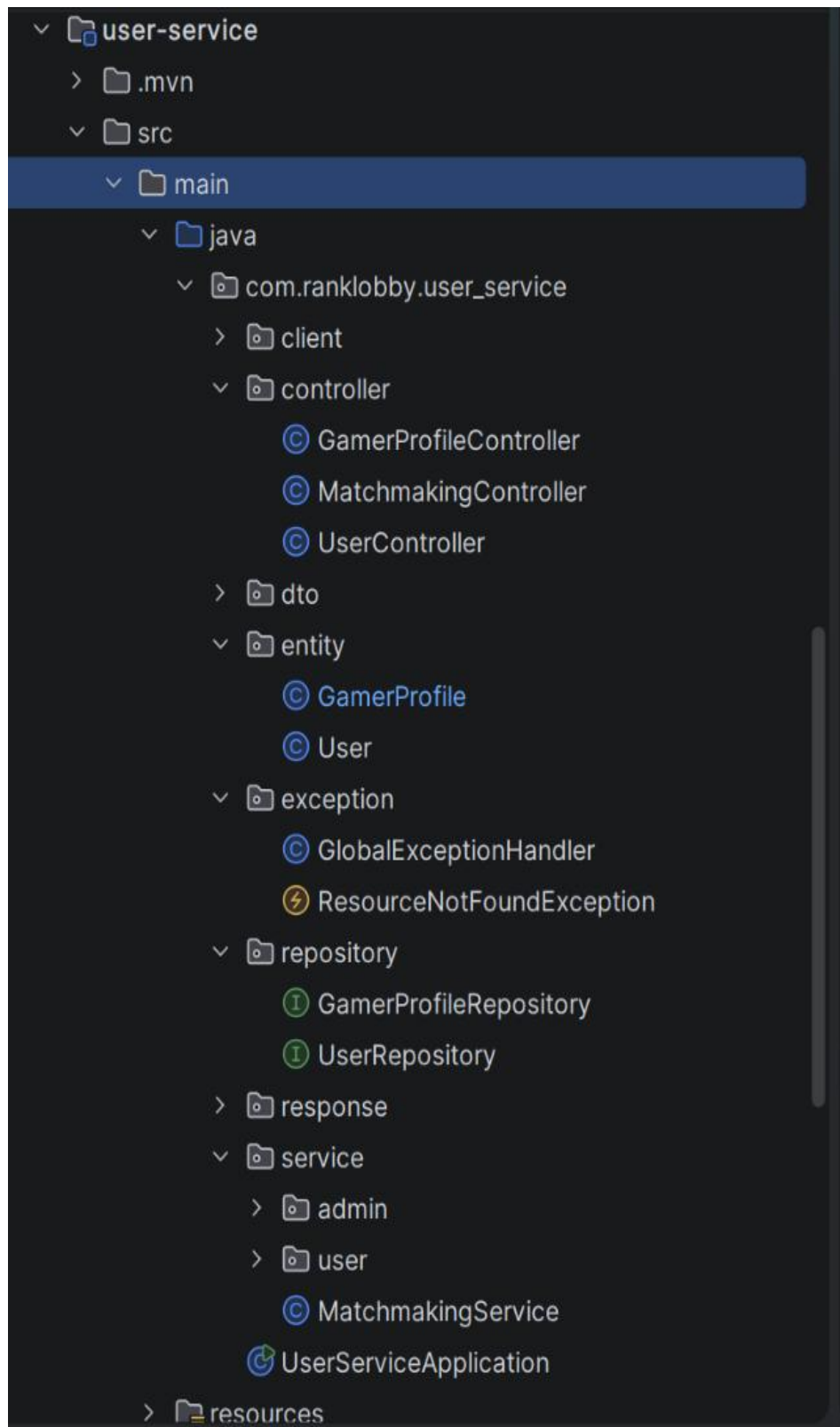
Chaque composant de l'application possède une architecture adaptée à son rôle.

Le webapp suit une architecture MVC composée de contrôleurs Spring MVC, de vues Thymeleaf et de clients OpenFeign permettant de communiquer avec les microservices.

Les microservices (user-service et social-service) suivent une architecture en couches composée de :

- Controller : réception des requêtes HTTP.
- Service : implémentation de la logique métier.
- Repository : accès aux bases de données.
- Entity / DTO : représentation et transfert des données.

Cette organisation facilite la maintenance, les évolutions et la séparation des responsabilités.



Arborescence du micro-service User avec son design pattern « MVC »

Sécurité

L'authentification est assurée par **Spring Security** au moyen d'une **session utilisateur**. Lors de la connexion, le CustomUserDetailsService interroge le user-service via **OpenFeign** afin de récupérer les informations du compte. Les mots de passe sont chiffrés avec **BCrypt** avant leur enregistrement en base de données. Deux rôles sont définis (USER et ADMIN) afin de restreindre l'accès aux fonctionnalités d'administration. Les comptes bannis ne peuvent pas se connecter à l'application.

```
public class SecurityConfig {  
  
    private final CustomUserDetailsService customUserDetailsService; 2 usages  
  
    public SecurityConfig(CustomUserDetailsService customUserDetailsService) { no usages 2 kael  
        this.customUserDetailsService = customUserDetailsService;  
    }  
  
    @Bean 2 kael *  
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {  
        http  
            .userDetailsService(customUserDetailsService)  
            .authorizeHttpRequests(AuthorizationManagerRequestMat... auth -> auth  
  
                //Not logged in  
                .requestMatchers(  
                    ...patterns: "/home",  
                    "/login",  
                    "/register",  
                    "/about",  
                    "/contact",  
                    "/legal",  
                    "/privacy-policy"  
                ).permitAll()  
                .requestMatchers(...patterns: "/css/**", "/js/**", "/images/**").permitAll()  
    }  
}
```

La classe SecurityConfig me permet, entre autre, de gérer les accès aux pages par authentification, autorisation, et les redirections de pages

Infrastructure technique

Les microservices s'enregistrent automatiquement auprès de **Consul** lors de leur démarrage. Celui-ci assure la découverte des services et permet aux microservices de communiquer entre eux sans avoir à configurer manuellement leurs adresses réseau. Le **Spring Cloud Config Server** est utilisé pour centraliser les fichiers de configuration des différents services. Durant les phases de développement, il peut être temporairement désactivé afin de simplifier les tests locaux, tout en conservant une architecture proche d'un environnement de production.

Flux de fonctionnement

Les principaux flux de l'application (inscription, authentification, création de profil et matchmaking) seront détaillés dans la section dédiée au plan de tests et jeu d'essai.

08 DÉVELOPPEMENT

Développement de l'interface utilisateur avec Thymeleaf

Le frontend de RankLobby est développé avec Thymeleaf afin de générer dynamiquement les pages HTML à partir des données du backend. Les différentes vues utilisent des fragments réutilisables (header, footer, ...), des expressions Thymeleaf, des conditions d'affichage et des formulaires liés aux objets Spring. Cette approche facilite la maintenance des interfaces tout en assurant une intégration naturelle avec Spring Boot.

```
29      <nav class="friend-filters" aria-label="Filtrer la liste d'amis">
30          <a th:href="@{/friends(filter='all')}}" class="filter-button active">
31              Tous
32          </a>
33
34          <a th:href="@{/friends(filter='pending')}}" class="filter-button">
35              En attente
36          </a>
37
38          <a th:href="@{/friends(filter='blocked')}}" class="filter-button">
39              Bloqués
40          </a>
41      </nav>
```

Thymeleaf permet de générer des templates (front) en y injectant les objets de mon backend

Authentification et autorisation

L'application distingue l'authentification et l'autorisation. L'authentification consiste à vérifier l'identité d'un utilisateur lors de sa connexion, tandis que l'autorisation détermine les ressources auxquelles il peut accéder selon son rôle. RankLobby utilise une authentification par session plutôt qu'un système JWT, ce choix étant plus adapté à une application web rendue avec Thymeleaf. Les mots de passe sont protégés avec BCrypt avant leur stockage en base de données. Le chargement des informations de connexion est assuré par le CustomUserDetailsService, qui récupère les données du compte depuis le user-service avant de les transmettre à Spring Security.

Fonctionnalités développées

Fonctionnalité	Développement réalisé
Inscription / Connexion	Création de compte et authentification sécurisée.
Profil utilisateur	Gestion du profil, avatar et message de statut.
Profil League of Legends (Gamer Profile)	Gestion du Riot ID, du rang, des LP et du rôle principal.
Matchmaking	Recherche de joueurs compatibles selon leur niveau.
Casual	Recherche de joueurs souhaitant jouer de manière détendue via une liste publique.
Fonctionnalités sociales	Invitations, amis, blocages, signalements et commentaires.
Administration	Gestion des utilisateurs et modération des signalements/commentaires

Difficultés rencontrées

Le principal défi rencontré concerne l'intégration de l'API Riot Games. Les limitations imposées aux clés développeur (renouvellement toutes les 24 heures) ainsi que les contraintes réglementaires empêchent la mise en place d'un système interne de calcul du MMR, ce qui limite certaines fonctionnalités de matchmaking avancé (côté couche service).

Le second défi a été de définir un périmètre réaliste pour cette première version.

Plusieurs fonctionnalités envisagées, comme un chat en temps réel basé sur MongoDB, ont volontairement été reportées afin de respecter les délais et le niveau technique attendu pour le projet.

Enfin, une autre difficulté rencontrée a été mon manque d'organisation en début de projet, qui m'a parfois conduit à vouloir brûler certaines étapes de conception avant de poursuivre le développement.

Perspectives d'évolution

Les principales évolutions prévues sont les suivantes :

Features :

- Mise en place d'un forum communautaire.
- Association des comptes Discord afin de faciliter la prise de contact entre joueurs.
- Mise à disposition d'une galerie d'avatars par défaut avec validation manuelle des avatars personnalisés.
- Intégration de l'historique des parties (saisie manuelle ou via l'API Riot Games).
- Calcul du taux de victoire (*winrate*) avec chaque coéquipier.
- Statistiques de progression du classement (LP gagnés/perdus, évolution par session, semaine ou mois).
- Ajout du jeu **Overwatch 2** et intégration de son API officielle.
- Ajout d'un rôle secondaire (*sub role*) dans le profil de jeu.
- Proposition automatique de bloquer et/ou signaler un joueur après un commentaire négatif.
- Prise en charge de nouveaux jeux compétitifs.
- Création d'un système de partage de clips.

Technologies :

- Mise en place d'un **chat en temps réel** (WebSocket).
- Déploiement des microservices avec **Kubernetes** afin de faciliter la scalabilité et l'orchestration des conteneurs.
- Intégration d'un **Circuit Breaker** (Resilience4j) pour améliorer la tolérance aux pannes lors des communications entre microservices.

09 DÉPLOIEMENT ET CONTENEURISATION

Cette partie présente la préparation du déploiement de l'application. L'objectif est de pouvoir exécuter l'application dans un environnement reproductible, en limitant les différences entre le poste de développement et un futur environnement d'exécution. Le projet s'appuie pour cela sur Docker et Docker Compose, afin de regrouper les microservices, les bases de données et les services techniques dans une même configuration.

Principe de conteneurisation avec Docker

Docker permet d'exécuter une application dans un conteneur isolé. Avant son exécution, l'application est intégrée dans une **image Docker**, qui regroupe tous les éléments nécessaires à son fonctionnement : le fichier exécutable de l'application (ici le .jar), les dépendances et l'environnement d'exécution. Lorsqu'une image est lancée, Docker crée un **conteneur**, c'est-à-dire une instance de cette image en cours d'exécution.

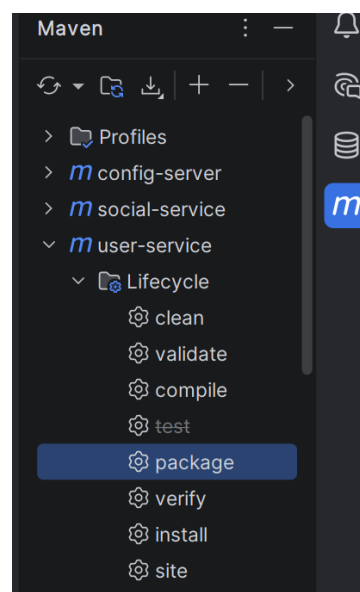
Dans le cas de RankLobby, chaque microservice Spring Boot est d'abord compilé sous forme de fichier .jar, puis intégré dans sa propre image Docker. Ce fonctionnement est particulièrement adapté à une architecture en microservices, puisque chaque composant : **webapp, user-service, social-service, Config Server, Consul, MySQL et MongoDB**

peut être exécuté dans un conteneur indépendant. Cette séparation facilite les tests, la maintenance et le déploiement de l'ensemble de l'application.

Construction des services Spring Boot

Avant de lancer les conteneurs, les services Spring Boot doivent être compilés avec Maven afin de générer les fichiers .jar. Ces fichiers sont ensuite utilisés par les Dockerfile de chaque service pour créer une image Docker exécutable.

mvn clean package (ou l'équivalent via l'IDE) :



Exemple des commandes données par Maven disponible sur l'IDE. Facilitant les cycles de vie

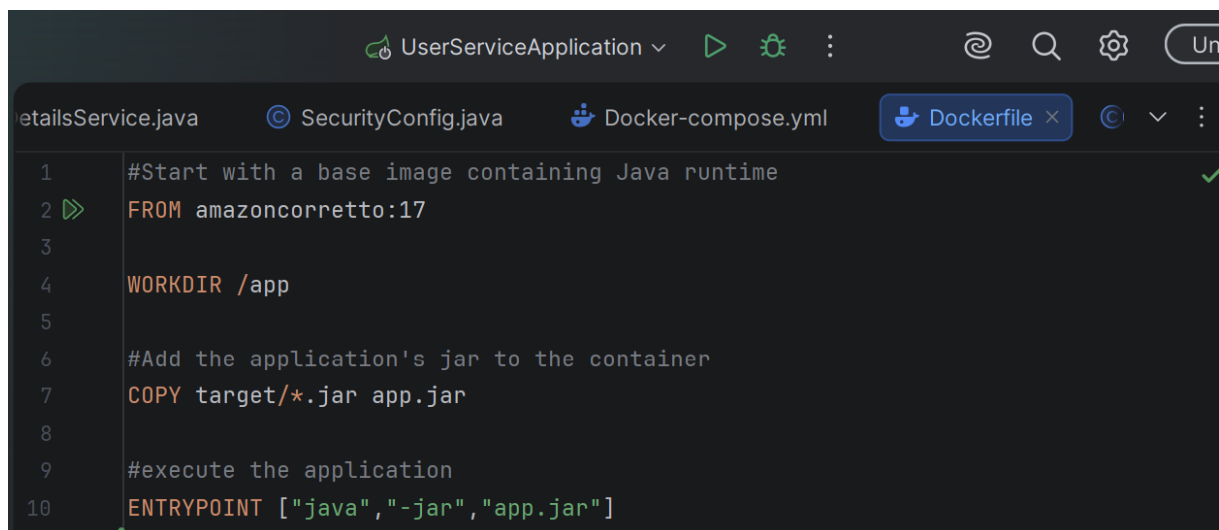
```

[INFO] --- jar:3.3.0:jar (default-jar) @ user-service ---
[INFO] Building jar: C:\Users\kaeve\Documents\MyProjects\RankLobby\user-service\target\user-service-0.0.1-SNAPSHOT.jar
[INFO]
[INFO] --- spring-boot:3.2.1:repackage (repackage) @ user-service ---
[INFO] Replacing main artifact C:\Users\kaeve\Documents\MyProjects\RankLobby\user-service\target\user-service-0.0.1-SNAPSHOT.jar with repackaged
[INFO] The original artifact has been renamed to C:\Users\kaeve\Documents\MyProjects\RankLobby\user-service\target\user-service-0.0.1-SNAPSHOT.jar
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 6.267 s
[INFO] Finished at: 2026-07-06T09:54:03+02:00
[INFO] -----

```

Le .jar est généré et rangé dans le dossier/target avec succès.

Le **Dockerfile** utilise ce fichier .jar pour construire une **image Docker**, qui contient l'application ainsi que son environnement d'exécution.



```

UserServiceApplication
1 #Start with a base image containing Java runtime
2 FROM amazoncorretto:17
3
4 WORKDIR /app
5
6 #Add the application's jar to the container
7 COPY target/*.jar app.jar
8
9 #execute the application
10 ENTRYPOINT ["java", "-jar", "app.jar"]

```

Dockerfile répété à chaque micro-service

Puis il faut build les images docker :

```

docker build -t ranklobby-user-service .
docker build -t ranklobby-social-service .
docker build -t ranklobby-webapp .

```

Une fois les images construites, elles peuvent être exécutées sous forme de conteneurs avec la commande docker run. Dans le cadre de RankLobby, le lancement est cependant réalisé avec **Docker Compose**, qui crée automatiquement les conteneurs, configure le réseau entre les différents services et démarre l'ensemble de l'environnement en une seule commande :

Dans une version automatisée, la commande docker compose up --build peut également reconstruire les images avant le lancement des services.

Chaine de construction :

1. Développement du code source Java.
2. Compilation avec Maven (mvn clean package) pour générer le fichier .jar.
3. Le Dockerfile copie le .jar dans une image Docker.
4. La commande docker build construit l'image Docker.
5. Docker Compose lance l'ensemble des images sous forme de conteneurs.

Orchestration avec Docker Compose

Docker Compose permet de décrire l'ensemble des services nécessaires à l'application dans un unique fichier docker-compose.yml. Celui-ci définit les images Docker, les ports, les variables d'environnement, les volumes, les réseaux et les dépendances entre les services. Il garantit notamment que **Config Server** et **Consul** sont démarrés avant les microservices qui en dépendent, ce qui facilite l'initialisation complète de l'application.

Pour RankLobby, Docker Compose permet de lancer les microservices applicatifs avec leurs dépendances techniques : MySQL pour les données relationnelles, MongoDB pour les avis, Consul pour la découverte des services et le Config Server pour la configuration centralisée.

Services et ports prévus

Service	Image / conteneur	Port	Rôle
webapp	ranklobby-webapp	8080	Interface Thymeleaf et point d'entrée utilisateur
user-service	ranklobby-user-service	8081	Utilisateurs, authentification et profils LoL
social-service	ranklobby-social-service	8082	Invitations, amis, signalements et avis
config-server	ranklobby-config-server	8071	Configuration centralisée
consul	consul	8500	Découverte et enregistrement des services
mysql	mysql:8	3306	Base relationnelle
mongo	mongo:7	27017	Base NoSQL pour les avis

Variables d'environnement et configuration

Les **variables d'environnement** sont des paramètres de configuration fournis à une application au moment de son exécution, sans être écrits directement dans le code source. Elles permettent de modifier facilement certains paramètres selon l'environnement utilisé (développement, test ou production), sans avoir à recompiler l'application.

Dans RankLobby, elles sont utilisées pour stocker les informations de configuration susceptibles d'évoluer, comme les identifiants de connexion MySQL, les URL des bases de données, l'adresse du Config Server, la configuration de Consul ou encore la clé d'accès à l'API Riot Games.

Cette approche présente plusieurs avantages :

- elle évite d'intégrer des informations sensibles directement dans le code source ;
- elle facilite le changement d'environnement (développement, recette, production) sans modifier l'application ;
- elle améliore la sécurité en limitant la diffusion des secrets.

Les variables sont regroupées dans un fichier `.env`, lu par Docker Compose lors du démarrage des conteneurs. Ce fichier n'est jamais push sur GitHub, car il peut contenir des identifiants, des mots de passe ou des clés d'API. Pour cette raison, je l'ai ajouté au fichier `.gitignore`, ce qui empêche son suivi par Git et évite sa publication accidentelle dans le dépôt.

```
1  #SQL
2  MYSQL_ROOT_PASSWORD=
3  MYSQL_DATABASE=ranklobby
4
5  MYSQL_DB_USERNAME=ranklobby_admin
6  MYSQL_DB_PASSWORD=
7
8  #RIOT API
9  RIOT_API_KEY=RGAPI-
```

Exemple de lancement

Le lancement complet de l'environnement se fait depuis la racine du projet, après compilation des services. La commande suivante permet de construire les images si nécessaire puis de démarrer les conteneurs définis dans le fichier Docker Compose.

```
PS C:\Users\kaeve\Documents\MyProjects\RankLobby> docker compose up --build
```

```
Docker-compose.yml x
1  services:
59
60  ▶  mysql:
61      image: mysql:8 # Uses the official MySQL version 8 Docker image
62      mem_limit: 700m # Limits container memory usage to 700 MB
63      container_name: ranklobby-mysql-db # Sets a fixed container name for easier identification
64      restart: always
65      environment: # Defines environment variables passed to the container
66          MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
67          MYSQL_DATABASE: ${MYSQL_DATABASE}
68          MYSQL_USER: ${MYSQL_USER}
69          MYSQL_PASSWORD: ${MYSQL_PASSWORD}
70      ports:
71          - "3306:3306" # Maps host port 3306 → container port 3306 (default MySQL port)
72      volumes: # Persists database data outside the container lifecycle
73          - ranklobby-mysql-data:/var/lib/mysql
74      networks: # Connects container to a custom Docker network
75          - ranklobby-network # Allows communication with other services on the same network
76      healthcheck:
77          test: [ "CMD", "mysqladmin", "ping", "-h", "localhost" ]
78          interval: 10s
79          timeout: 5s
80          retries: 10
81          start_period: 10s
82
83  ▶  user-service:
84      image: ranklobby-user-service
85      container_name: ranklobby-user-service
86      mem_limit: 700m
87      restart: on-failure
88
```

Docker-compose.yml placé à la racine du dossier.

Enfin, il suffit d'exécuter la commande :

`docker compose up --build`

pour démarrer les conteneurs.

Documentation du déploiement

En complément de la conteneurisation, une documentation a été réalisée afin de faciliter l'installation et la prise en main du projet. Un **README** accompagne le dépôt GitHub et présente les prérequis, les technologies utilisées, l'architecture de l'application, les variables d'environnement, les commandes de compilation et les étapes nécessaires au déploiement des microservices.

Les API REST sont également documentées grâce à **Swagger**. Cette interface permet de consulter les endpoints disponibles, leurs paramètres, les réponses attendues ainsi que de tester directement les requêtes HTTP depuis un navigateur. Elle facilite le développement, les phases de test et la maintenance de l'application.

10 PLAN DE TESTS ET JEU D'ESSAI

Cette phase a permis de vérifier le bon fonctionnement des fonctionnalités développées avant leur intégration dans l'application web. Les tests réalisés garantissent que les échanges entre la webapp et les microservices sont conformes aux règles métier et que les données sont correctement enregistrées en base de données.

Outils utilisés

Outil	Utilisation
Navigateur	Tests fonctionnels
Swagger	Tests des endpoints
Postman	Tests API
MySQL Workbench	Contrôle SQL
MongoDB Compass	Contrôle MongoDB

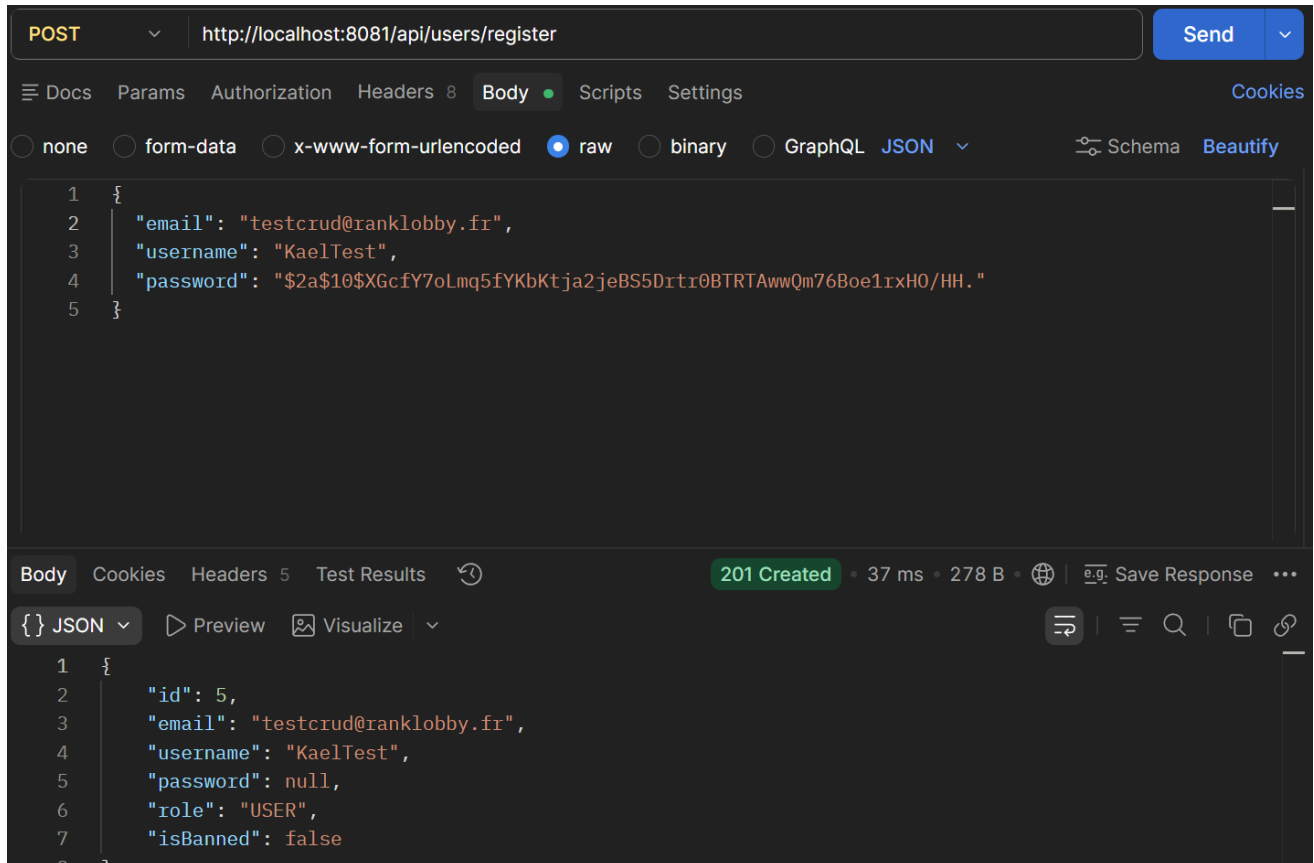
Tests de l'API avec Postman

Postman a été utilisé pour tester les endpoints REST indépendamment de l'interface graphique. Cette approche permet de vérifier rapidement les requêtes HTTP, les paramètres, les codes de retour et les données échangées entre le client et le serveur. Les requêtes sont regroupées dans une collection afin de pouvoir être rejouées facilement pendant les phases de développement et de démonstration.

Démonstration d'un CRUD côté user-service

Les tests couvrent les quatre opérations principales d'une API REST :

- Create : création d'un utilisateur.



Test de l'endpoint POST register

Remarque : lors des tests réalisés avec Postman sur le user-service (port 8081), un mot de passe déjà haché avec BCrypt est transmis dans la requête. En fonctionnement normal, ce hachage est effectué par la webapp avant l'appel au microservice. Ce choix permet de tester directement l'API du user-service indépendamment de l'interface utilisateur.

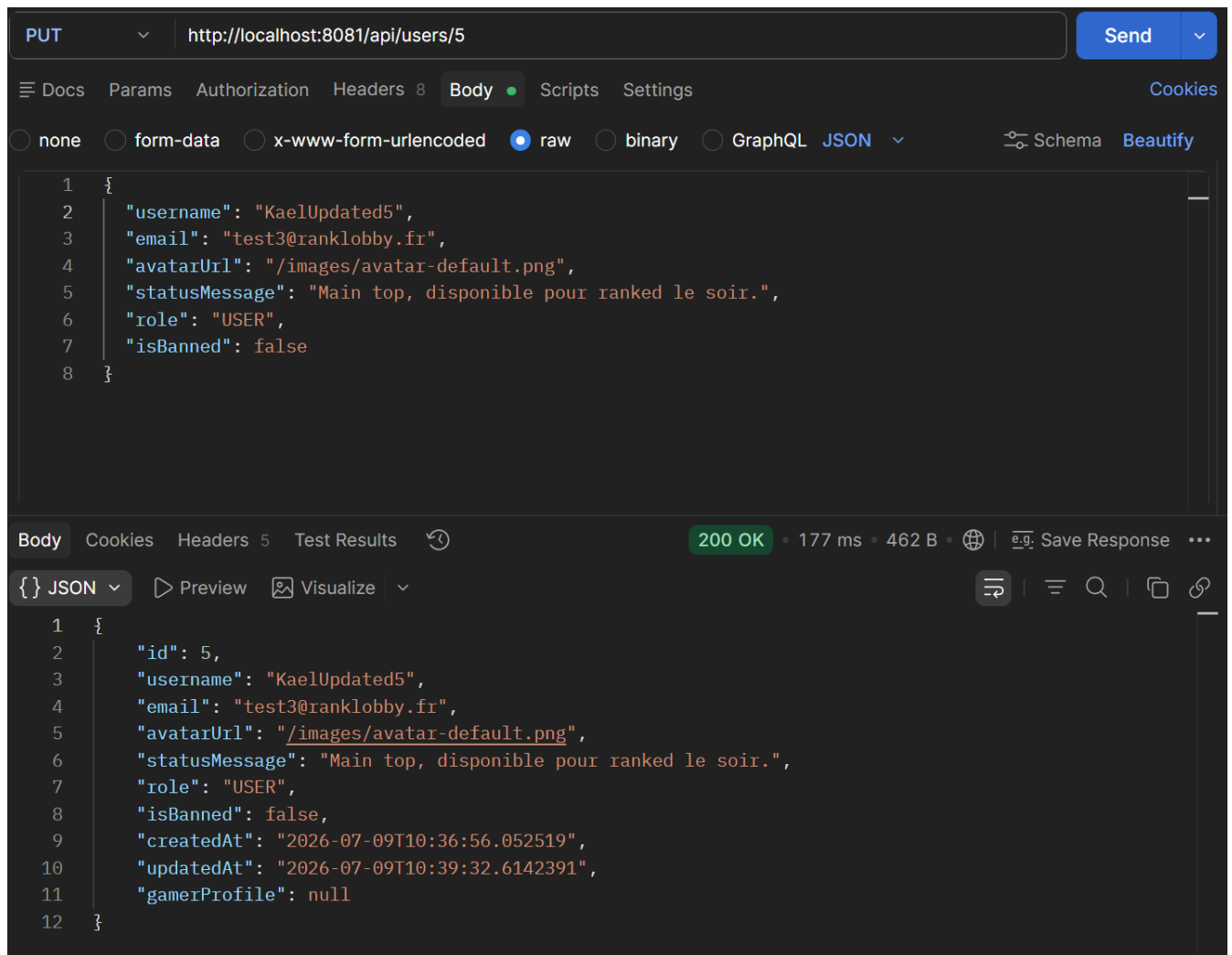
- Read : consultation des informations d'un utilisateur.

The screenshot shows a REST client interface with a GET request to `http://localhost:8081/api/users/5`. The response is a 200 OK status with a response time of 18 ms and a body size of 273 B. The response body is displayed in JSON format:

```
1 {
2   "id": 5,
3   "email": "testcrud@ranklobby.fr",
4   "username": "KaelTest",
5   "password": null,
6   "role": "USER",
7   "isBanned": false
8 }
```

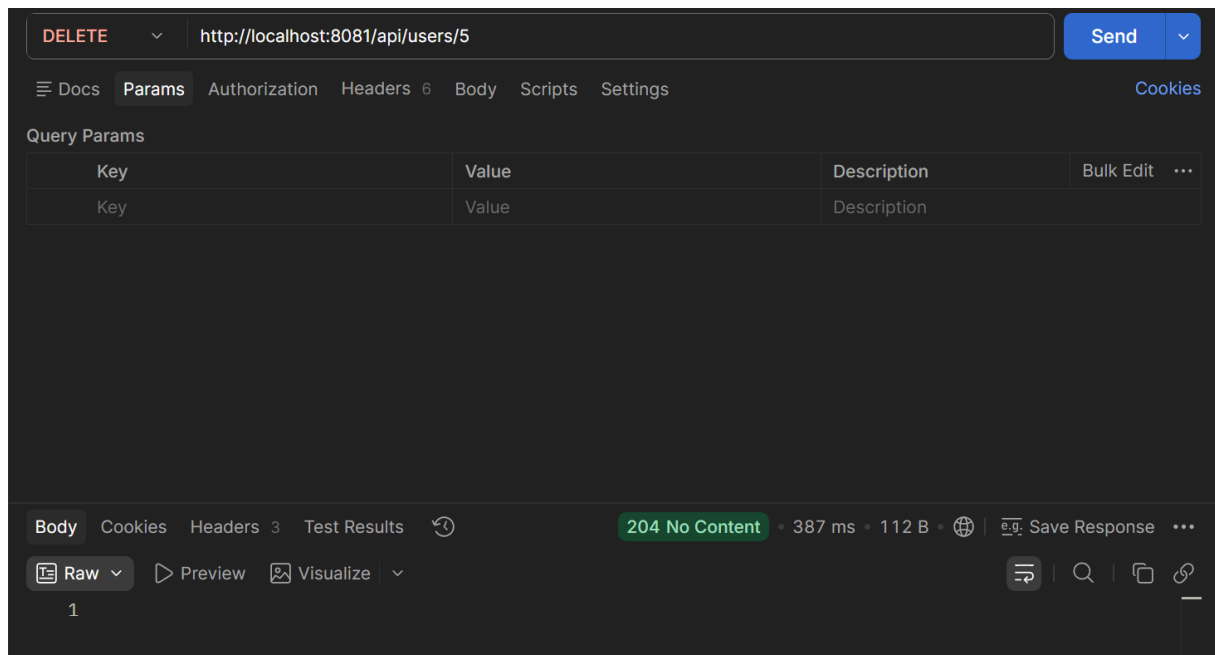
Test de l'endpoint GET user/{id}

- Update : modification des données.



Test de l'endpoint PUT update by id. Patch aurait été utilisé en cas de modification de certains champs uniquement.

Delete : suppression d'un utilisateur



Test de l'endpoint DELETE by id

Chaque opération est vérifiée à l'aide des codes de retour HTTP (200, 201, 400, 404...) ainsi que par le contrôle des données enregistrées dans la base MySQL.

Vérification en base de données

Après chaque requête POST, PUT ou DELETE, la base MySQL est contrôlée afin de vérifier que les données correspondent au résultat attendu. Une attention particulière est portée au mot de passe, qui est stocké sous forme de hash BCrypt généré par la webapp. Pour cela, j'utilise MySQL WorkBench et MongoDB Compass qui sont des GUI (interface utilisateur graphique).

The screenshot shows the MySQL Workbench interface. On the left, the 'SCHEMAS' pane displays the 'ranklobby' database with its tables: friendship, friend_invitation, gamer_profile, user_block, report, and users. The 'users' table is selected, and a query is executed: `SELECT * FROM ranklobby.users;`. The 'Result Grid' shows two rows of user data.

	id	avatar_url	created_at	email	is_banned	password	role	status_message	updated_at	username
1	MAKS		2026-07-06 13:51:36.217369	kael@test.fr	0	\$2a\$10\$Hc9KwAeWBFf8qT8w3JF4dL4vOxLkGvzqwOSw5YpcOsrSm...	USER	MAKS	MAKS	Kael
2	MAKS		2026-07-06 14:58:58.647325	test@rankl...	0	\$2a\$10\$Gcf77Lmq5fYKkjaZyeBSDrtr08TRTAwwQm76BoeIrxHO...	USER	MAKS	MAKS	KaelTest

MySQL Workbench pour vérifier lire tous les users.

Test unitaires

En exemple de test unitaire, réalisé sur le service de gestion des invitations d'amis (FriendInvitationService). Il vérifie qu'un utilisateur ne peut pas s'envoyer une invitation à lui-même. Cette règle métier est testée indépendamment de l'interface utilisateur et de la base de données grâce à Mockito, qui permet de simuler le repository et de vérifier qu'aucune invitation n'est enregistrée lorsqu'une `IllegalArgumentException` est levée.

```

16 class FriendInvitationServiceTest {
17     @Mock 1 usage
18     private FriendInvitationRepository friendInvitationRepository; //Mock = faux repository créé par Mockito, aucune base MySQL n'est utilisée.
19
20     @InjectMocks 1 usage
21     private FriendInvitationService friendInvitationService;
22
23     @Test new *
24     void sendInvitation_shouldThrowException_whenSenderAndReceiverAreSame() {
25
26         FriendInvitationRequestDto dto = new FriendInvitationRequestDto();// Création d'une fausse requête d'envoi d'invitation
27
28         dto.setSenderId(1L); // L'utilisateur ayant l'ID 1 essaye d'envoyer une invitation à lui-même (même Id).
29         dto.setReceiverId(1L);
30
31         //Bien que cette situation ne doive pas être permise côté front, le backend doit également invalider cette situation.
32         assertThrows( //La couche service doit empêcher que cela soit possible en retournant une "IllegalArgumentException".
33             IllegalArgumentException.class,
34             () -> friendInvitationService.sendInvitation(dto)
35         );
36
37         // On vérifie également qu'il n'y a pas eu d'accès à la BDD via "friendInvitationRepository".
38         verify(friendInvitationRepository, never())
39             .save(org.mockito.ArgumentMatchers.any());
40     }

```

Run FriendInvitationServiceTest

✓ FriendInvitationServiceTest 877 ms ✓ 1 test passed 1 test total, 877 ms

Test unitaire pour FriendInvitation

Exemple de jeux d'essai

Test	Entrée	Résultat attendu	Statut
Inscription	Email valide + mot de passe	Utilisateur créé, hash BCrypt enregistré	OK
Connexion	Identifiants valides	Connexion réussie et redirection vers le profil	OK
Consultation profil	Utilisateur connecté	Informations affichées	OK
CRUD utilisateur	Requêtes Postman	Réponse HTTP conforme et données mises à jour	OK
FriendInvitation	Création d'une invitation	Test unitaire validé	OK

Les jeux d'essai permettent de vérifier que les fonctionnalités principales de l'application répondent bien aux besoins attendus. Ils couvrent ici le parcours utilisateur principal : inscription, connexion, consultation du profil, gestion des utilisateurs et création d'une invitation d'ami.

Conclusion

Les tests réalisés permettent de valider les principales fonctionnalités développées dans le cadre du projet. Ils couvrent à la fois les échanges API, la persistance des données en base ainsi que la validation d'une règle métier au moyen d'un test unitaire.

11 CONCLUSION

Le projet **RankLobby** m'a permis de mettre en pratique l'ensemble des compétences du titre professionnel **Développeur Web et Web Mobile**, depuis la conception fonctionnelle et visuelle jusqu'au développement d'une application web sécurisée reposant sur une architecture Spring Boot.

Ce projet m'a permis d'approfondir ma maîtrise de **Java 17**, **Spring Boot**, **Spring Security**, **Spring Data JPA**, **MongoDB**, **Thymeleaf** et **Docker**, tout en découvrant le fonctionnement d'une architecture reposant sur plusieurs services communiquant entre eux grâce à **OpenFeign**, **Consul** et **Spring Cloud Config Server**. J'ai également consolidé mes compétences en modélisation de bases de données avec la méthode **Merise**, en conception d'interfaces **mobile-first** et en utilisation de Git pour le suivi du développement.

L'application propose déjà les principales fonctionnalités attendues : création de compte, authentification sécurisée, gestion des profils utilisateurs et des profils de jeu, recherche de joueurs, système d'amis, blocage, signalement, administration ainsi que la gestion des avis entre joueurs. Certaines fonctionnalités envisagées, comme le chat en temps réel, les forums ou le partage de clips, n'ont volontairement pas été développées dans cette première version afin de respecter les délais de réalisation.

Au-delà de l'aspect technique, ce projet m'a appris à concevoir une application complète en réfléchissant à son architecture, à la sécurité des données, à la maintenabilité du code et à l'expérience utilisateur. Il constitue une étape importante dans mon parcours et représente une base solide que je souhaite continuer à faire évoluer après l'obtention du titre professionnel.